Sineth Jayasundera

Professor Kambhampaty

Cmpsc 132

April 28, 2026

# Tic-Tac-Toe Final Project Program

• Project description

As per the instructions given to me, using a python program, I created a two-player game of tic-tac-toe. Players take turns inputting the position of where they want to place their piece. From which, one of their pieces, either an 'x' or an 'o' is printed with an updated board in the terminal. As typically with the game Player 1 starts with the 'x' pieces and Player 2 follows with 'o' pieces.

The program starts by asking player 1 to make the first move. After this the game goes accordingly until a win or draw is reached. From there, the winner is determined and the number of rounds, current scores for player 1 and player 2, and the number of wins, loses, and draws are all displayed. After each round a play again option is enabled, and the next round begins.

• Approach, Data Structures, and logic used

I used a function-based approach to create the game. Deciding to deal with user input first under the **play()** function (the function that would be called to play the game) I set a cap of 9 rounds with a while loop, and allowed player 1 and player 2 to input the positions they wanted the pieces to be in via **x** and **y** variable for inputs for the row and column. I ran a modulus check on the value of the round to determine which player needed to play. I decided to let users input the row and column as numbers from 0-2 where, for example, an input of row – 0 col – 2 would corelate to the top right position of the board.

Thus, the function **input_Validation()** checks the input to see if it is only 1 character, takes the row and column value and uses the **try and except** to convert the numbers into an integer safely and check if the value is between 0-2. If this returns false, **play()** uses a while loop to request another input from that player.

The function **print_board()** exploits the modulus check on the round in **play()** where a Boolean (**player_1)** is flipped every round. if **player_1** is true then an 'x' is drawn on the board otherwise it is an 'o'. As requested in the implementation details, the board is a list of lists for each row. The empty **Board** has each position filled with a '.' To

represent an empty space. Thus, taking the **x and y** input, I edit the **Board** and print it every round. This function also checks if a position has already been written in (the position already has a 'o' or an 'x'). If it was been written in the **play()** function will redo that round and let the player choose another position.

Within the while loop in **play()** on top of iterating the round, the loop also monitors the **win_Check()** function. **Win_Check()** has a nested list called possible_checks which contains lists of all possible positions that could be used to win. The function then iterates through the list and checks every permutation. Assuming a win exists, it tries to disprove this case by every having position checked not be the same value ('x' or 'o') or if any position has a '.', indicative of an empty row/col. The function will return False if either of these cases are met. If **win_Check()** returns True, the while loop is exited and the player_1 Boolean is checked to see which players turn it was and thereby which player won the game.

Finally, **main()** is used to call play for every game the 2 players have. It keeps track of the number of games, and the scores of each player. The stats are shown after each game. Then the players are asked if they again with a y/n input to play another game or quit the program.

• Challenges faced

I initially made the program as a board with a single list. Changing it to a nested list required me to rewrite a lot of the functions.

I also had difficulty working out the logic for the check function. I tried a few recursive methods to solve the problem until a realized there were a limited number of possible ways to win.

• Possible improvements

I think there is definitely some excess code in play() in regards to taking input positions that could be placed in a separate function.

I also believe that there should be some way to make user input easier that finding the row and column for every input.
Possibly one could use pygame to make user input seamless and make the game very fun.
I think the '.' I used for empty space in each position is not needed, it could be kept as a 'None' value. I had initially believed that I might need there to be a placeholder for empty space but I was wrong.